Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 16

Text Generation

Building a GPT from the parts we already have, then fine-tuning a billion-parameter model into a chatbot — and being honest about what none of it fixes.

Duration 4 hours (3 sessions)

Instructor Prof. Bambang Riyanto Trilaksono · Rahman Indra Kesuma, S.Kom., M.Cs.

Source Chollet & Watson, Deep Learning with Python 3e — chapter 16

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. Trace the line from **LSTM in 1997 to ChatGPT**, and name the three ingredients GPT combined.
2. Explain why GPT is **decoder-only**, and what that costs and buys.
3. Build a **mini-GPT**: data pipeline, tied embeddings, warmup schedule, and a logits-based loss.
4. Implement and compare **greedy, random, temperature, and top-K** sampling, and say what each failure mode looks like.
5. Explain **key-value caching** and why generation is slow without it.
6. **Instruction fine-tune** a pretrained LLM, and use **LoRA** to make it fit in accelerator memory.
7. Describe **RLHF, multimodal input, RAG, and chain-of-thought training** well enough to judge which one a given problem needs.
8. State the **structural limits**: hallucination, prompt sensitivity, energy cost, and a pretraining-data supply that is running out.

BOOK
Chapter 16 — full text

NOTEBOOK
01_mini_gpt_data_pipeline.ipynb

NOTEBOOK
02_training_mini_gpt.ipynb

NOTEBOOK
03_sampling_strategies.ipynb

NOTEBOOK
04_gemma_generation.ipynb

NOTEBOOK
05_instruction_tuning_with_lora.ipynb

NOTEBOOK
All chapter 16 notebooks

REPO
Author's official notebook

01

A brief history of sequence generation

Twenty-five years from a recurrent cell to a consumer product.

SECTION 16.1

This was a niche subtopic until very recently

Sometimes a good idea takes fifteen years to get started.

Douglas Eck applied LSTM to **music generation** in 2002, became a researcher at Google Brain, and in 2016 started Magenta. Alex Graves pioneered recurrent networks for new kinds of sequence data — his 2013 work generating human-like **handwriting** from pen-position timeseries is seen by many as the turning point.

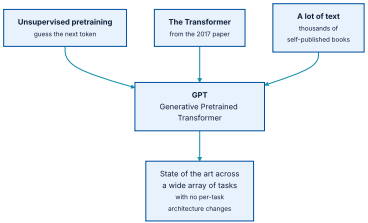
The closest computers get to dreaming

> *"Generating sequential data is the closest computers get to dreaming."*
>
> Alex Graves, in a commented-out line of a 2013 LaTeX file on arXiv

The line was never meant to be read. Chollet cites it as a significant inspiration behind starting Keras — which is to say that the framework this course is taught in has a remark hidden in a preprint somewhere in its lineage.

Worth remembering when the field feels purely industrial. Much of what follows in this chapter came out of curiosity long before it came out of a product plan.

GPT combined three things, and invented none of them



Generative Pretrained Transformer — three known ingredients, one surprising result.

GPT came with no modelling or training advancements. What was interesting is that such a general setup beat more involved, task-specific techniques — no complex text normalization, no per-benchmark architecture. Just **pretraining data and compute**.

Then OpenAI scaled it with single-minded focus

MODEL	YEAR	PARAMETERS	TRAINING TOKENS
GPT-1	2018	117 million	1 billion
GPT-2	2019	1.5 billion	more than 10 billion
GPT-3	2020	175 billion	around half a trillion

The architecture changed only slightly across all three. What changed was the size of everything else — and with each leap in scale, the quality of the generative output shot up substantially.

That is roughly a **1,500-fold** increase in parameters and a **500-fold** increase in tokens over four years, for the same design.

Three models, three different ways of being used

GPT-1: fine-tune it

Generative capability was a **by-product**. Evaluation was by attaching a dense classification head — exactly what we did to RoBERTa in chapter 15.

GPT-2: show it examples

Prompt with a handful of worked examples and get quality output **with no fine-tuning at all**. This is *few-shot learning*.

GPT-3: just describe it

Examples were often unnecessary. A plain text description of the problem plus the input frequently sufficed.

Few-shot learning means teaching a model a new problem with only a handful of supervised examples — **far too few for standard gradient descent**. Nothing is being trained; the capability was already in the pretrained distribution.

What a GPT-2 prompt looked like

OUTPUT

```
Translate English to French:

sea otter => loutre de mer
peppermint => menthe poivrée
plush giraffe => peluche girafe
cheese =>
```

By GPT-3, this was often enough:

OUTPUT

```
Translate English to French:

cheese =>
```

Nothing in the model was updated between the two. The difference is entirely in **what the larger pretrained distribution already contains**.

GPT-3's three unsolved problems, stated in 2020

Hallucination

Output veers from accurate to **completely false with zero indication**. Nothing in the output signals which is which.

Prompt sensitivity

Seemingly minor rewording triggers **large jumps** in performance, up or down. Behaviour is hard to predict.

No adaptation

The model cannot adapt to problems that were not **extensively featured** in its training data.

These were listed as open problems in 2020 and are **still open**. Everything later in this chapter — instruction tuning, RLHF, RAG, chain-of-thought — reduces their severity. **None of them removes the problem.**

Nevertheless GPT-3's output was good enough to become the basis for ChatGPT, the first widespread consumer-facing generative model.

02

Training a mini-GPT

Forty-one million parameters, a billion tokens, and the exact blueprint everyone else is using.

SECTION 16.2

The data: 1% of a colossal clean crawled corpus

We use **C4**, the Colossal Clean Crawled Corpus released by Google in 2020. At 750 GB it is far more than we can train on, so we take under 1%.

listing 16.1 PYTHON

```python
import keras
import pathlib

extract_dir = keras.utils.get_file(
    fname="mini-c4",
    origin=(
        "https://hf.co/datasets/mattdangerw/mini-c4/resolve/main/mini-c4.zip"
    ),
    extract=True,
)
extract_dir = pathlib.Path(extract_dir) / "mini-c4"
```

Fifty shards, each about 75 MB of raw text. One document per line, newlines escaped.

This is the most compute-intensive chapter in the book

~6 hours
mini-GPT on a free Colab T4

~1 hour
the whole chapter on an A100

1
runtime restart needed mid-notebook

You will need to restart the Colab runtime partway through to free GPU memory before loading the larger pretrained model.

> You can always **read through** the expensive `fit()` calls and edit down the step count for quick experimentation. The learning is in the code, not in waiting for it.

And if you are running this a few years from now, there is a good chance these examples are child's play to the hardware in front of you.

SECTION 16.2

What one document looks like

OUTPUT

```
>>> with open(extract_dir / "shard0.txt", "r") as f:
>>>     print(f.readline().replace("\\n", "\n")[:100])
Beginners BBQ Class Taking Place in Missoula!
Do you want to get better at making delicious BBQ? You
```

Crawled web text: a headline, a newline, marketing copy. This is what the model's distribution will be made of, and it is worth holding that image when the output later looks like a **forum post that will not end**.

SentencePiece: the same BPE, in C++

listing 16.2 PYTHON

```python
import keras_hub
import numpy as np

vocabulary_file = keras.utils.get_file(
    origin="https://hf.co/mattdangerw/spiece/resolve/main/vocabulary.proto",
)
tokenizer = keras_hub.tokenizers.SentencePieceTokenizer(vocabulary_file)
```

OUTPUT

```
>>> tokenizer.tokenize("The quick brown fox.")
array([ 450, 4996, 17354, 1701, 29916, 29889], dtype=int32)
>>> tokenizer.detokenize([450, 4996, 17354, 1701, 29916, 29889])
"The quick brown fox."
```

A premade vocabulary of **32,000 terms**. KerasHub wraps the library as a Keras layer, so it composes with `tf.data`.

GPT does not respect document boundaries

Instead, a boundary is marked with a special token:

OUTPUT

```
<|endoftext|>
```

The model learns what a document boundary means from the token itself, rather than from how the data was cut.
Simplicity in the pipeline, paid for with one vocabulary entry.

> This is a recurring pattern in LLM engineering: when a structural constraint is expensive to enforce in data, **encode it as a token and let the model learn it**.

The pipeline, with interleaved shards

listing 16.3 PYTHON

```python
import tensorflow as tf

batch_size = 128
sequence_length = 256
suffix = np.array([tokenizer.token_to_id("<|endoftext|>")])

def read_file(filename):
    ds = tf.data.TextLineDataset(filename)
    ds = ds.map(lambda x: tf.strings.regex_replace(x, r"\\n", "\n"))
    ds = ds.map(tokenizer, num_parallel_calls=8)
    return ds.map(lambda x: tf.concat([x, suffix], -1))

files = [str(file) for file in extract_dir.glob("*.txt")]
ds = tf.data.Dataset.from_tensor_slices(files)
ds = ds.interleave(read_file, cycle_length=32, num_parallel_calls=32)
ds = ds.rebatch(sequence_length + 1, drop_remainder=True)
ds = ds.map(lambda x: (x[:-1], x[1:]))
ds = ds.batch(batch_size).prefetch(8)
```

`rebatch(sequence_length + 1)` windows the token stream into even 257-token samples; `x[:-1], x[1:]` is the offset-by-one split from chapter 15, unchanged.

Just under a billion tokens

29,373
batches

128 × 256
samples × tokens per batch

~0.96 billion
tokens of training data

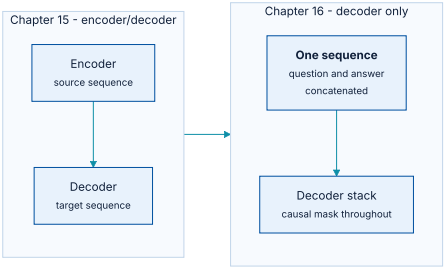
PYTHON

```
num_batches = 29373
num_val_batches = 500
num_train_batches = num_batches - num_val_batches

val_ds = ds.take(num_val_batches).repeat()
train_ds = ds.skip(num_val_batches).repeat()
```

The batch count is not free to obtain: `ds.reduce(0, lambda c, _: c + 1)` will count it, but tokenizing a dataset this size takes **several minutes on a fast CPU**. The `prefetch(8)` at the end of the pipeline is what keeps the GPU from waiting on it.

SECTION 16.2.1

GPT throws away the encoder



A question and its answer become one sequence, embedded by the same parameters.

A decoder-only model can still handle sequence-to-sequence problems like question answering — but the question and answer must be **combined into a single sequence**, and question tokens are treated no differently from answer tokens.

What the decoder-only bet costs, and what it buys

The cost: less expressive inputs

Given *"Where is the capital of France?"*, the representation of **Where** cannot attend to **capital** or **France**. Even the input is processed one-directionally.

The gain: no curation at all

No need for datasets of input/output **pairs**. Everything is a single sequence, so you can train on any text on the internet, at any scale you can afford.

This is the trade that decided the field. Expressivity was sacrificed for **an unbounded supply of training data**, and the data won.

The decoder block, minus cross-attention

listing 16.4 PYTHON

```python
class TransformerDecoder(keras.Layer):
    def __init__(self, hidden_dim, intermediate_dim, num_heads):
        super().__init__()
        key_dim = hidden_dim // num_heads
        self.self_attention = layers.MultiHeadAttention(
            num_heads, key_dim, dropout=0.1
        )
        self.self_attention_layernorm = layers.LayerNormalization()
        self.feed_forward_1 = layers.Dense(intermediate_dim, activation="relu")
        self.feed_forward_2 = layers.Dense(hidden_dim)
        self.feed_forward_layernorm = layers.LayerNormalization()
        self.dropout = layers.Dropout(0.1)
```

Chapter 15 used a **single** Transformer layer, so one dropout at the end of the model sufficed. Here we stack **eight**, and regularisation inside each block matters.

Its forward pass has only two stages now

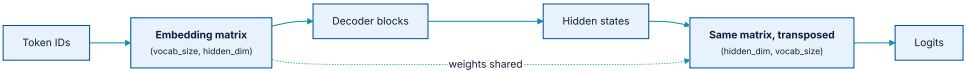
listing 16.4 - call PYTHON

```
    def call(self, inputs):
        residual = x = inputs
        x = self.self_attention(query=x, key=x, value=x, use_causal_mask=True)
        x = self.dropout(x)
        x = x + residual
        x = self.self_attention_layernorm(x)

        residual = x
        x = self.feed_forward_1(x)
        x = self.feed_forward_2(x)
        x = self.dropout(x)
        x = x + residual
        x = self.feed_forward_layernorm(x)
        return x
```

`use_causal_mask=True` is doing all the work that made the training objective valid in chapter 15. **It is the only thing standing between this model and reading its own labels.**

Tying the input and output projections

They are transposes of each other in shape. It turns out that making them transposes of each other in **value** is a good idea — the final projection is a *reverse embedding*, mapping hidden space back to token space.



One matrix, used forward at the input and transposed at the output.

A `reverse` argument is all it takes

listing 16.5 PYTHON

```python
from keras import ops

class PositionalEmbedding(keras.Layer):
    def __init__(self, sequence_length, input_dim, output_dim):
        super().__init__()
        self.token_embeddings = layers.Embedding(input_dim, output_dim)
        self.position_embeddings = layers.Embedding(sequence_length, output_dim)

    def call(self, inputs, reverse=False):
        if reverse:
            token_embeddings = self.token_embeddings.embeddings
            return ops.matmul(inputs, ops.transpose(token_embeddings))
        positions = ops.cumsum(ops.ones_like(inputs), axis=-1) - 1
        embedded_tokens = self.token_embeddings(inputs)
        embedded_positions = self.position_embeddings(positions)
        return embedded_tokens + embedded_positions
```

With a 32,000-word vocabulary and 512 hidden dimensions, this saves **16 million parameters** — a substantial share of a 41-million-parameter model.

Eight blocks, and the model is done

listing 16.6 PYTHON

```python
keras.config.set_dtype_policy("mixed_float16")

vocab_size = tokenizer.vocabulary_size()
hidden_dim = 512
intermediate_dim = 2056
num_heads = 8
num_layers = 8

inputs = keras.Input(shape=(None,), dtype="int32", name="inputs")
embedding = PositionalEmbedding(sequence_length, vocab_size, hidden_dim)
x = embedding(inputs)
x = layers.LayerNormalization()(x)
for i in range(num_layers):
    x = TransformerDecoder(hidden_dim, intermediate_dim, num_heads)(x)
outputs = embedding(x, reverse=True)
mini_gpt = keras.Model(inputs, outputs)
```

41 million parameters — large for this book, and tiny against models today that range from a couple of billion to trillions. `mixed_float16` trades some numerical fidelity for speed; chapter 18 explains it properly.

SECTION 16.2.2

Training a large Transformer is famously finicky

A trick that works well: ease **linearly** into the full learning rate over a number of warmup steps, so the earliest updates are small.

listing 16.7 PYTHON

```python
class WarmupSchedule(keras.optimizers.schedules.LearningRateSchedule):
    def __init__(self):
        self.rate = 2e-4
        self.warmup_steps = 1_000.0

    def __call__(self, step):
        step = ops.cast(step, dtype="float32")
        scale = ops.minimum(step / self.warmup_steps, 1.0)
        return self.rate * scale
```

A thousand steps of ramp, then a flat `2e-4`. Plot it before training — a schedule that is wrong is invisible in the loss curve until it is far too late.

One pass over a billion tokens, in eight epochs

listing 16.8 PYTHON

```python
num_epochs = 8
steps_per_epoch = num_train_batches // num_epochs
validation_steps = num_val_batches

mini_gpt.compile(
    optimizer=keras.optimizers.Adam(schedule),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)
mini_gpt.fit(
    train_ds,
    validation_data=val_ds,
    epochs=num_epochs,
    steps_per_epoch=steps_per_epoch,
    validation_steps=validation_steps,
)
```

We use **3× fewer parameters** than GPT-1 and **100× fewer training steps**. Even so, this is the most computationally expensive training run in the entire book.

What is a logit?

The common term for an unnormalized log probability is a **logit**, and logits are easier to work with when generating text — as the sampling section will show.

Softmax in the model

`Dense(n, activation="softmax")` plus `SparseCategoricalCrossentropy()`. The model outputs probabilities.

Softmax in the loss

`Dense(n)` plus `SparseCategoricalCrossentropy(from_logits=True)`. The model outputs logits — **more numerically stable, and easier to sample from**.

SECTION 16.2.2

36% — and the model is deliberately undertrained

> The validation loss is **still ticking down** after every epoch. This model is not converged and we know it — unsurprising with a hundred times fewer training steps than GPT-1. Training longer would help; it would also cost time and money.

Being explicit about an undertrained model is itself a discipline. Most published curves stop where the budget ran out, not where the model stopped improving.

03

Decoding and sampling strategies

The model gives you a distribution. What you do with it is a separate design.

The naive generation loop

listing 16.9 PYTHON

```python
def generate(prompt, max_length=64):
    tokens = list(ops.convert_to_numpy(tokenizer(prompt)))
    prompt_length = len(tokens)
    for _ in range(max_length - prompt_length):
        prediction = mini_gpt(ops.convert_to_numpy([tokens]))
        prediction = ops.convert_to_numpy(prediction[0, -1])
        tokens.append(np.argmax(prediction).item())
    return tokenizer.detokenize(tokens)
```

Correct, readable — and it takes **minutes** to produce 64 tokens. That is the puzzle to solve before anything else.

SECTION 16.2.3

What it produces, and why it is slow

OUTPUT

```
>>> prompt = "A piece of advice"
>>> generate(prompt)
A piece of advice, and the best way to get a feel for yourself is to get a sense
of what you are doing.
If you are a business owner, you can get a sense of what you are doing. You can
get a sense of what you are doing, and you can get a sense of what
```

Two separate problems, and it is worth naming them apart. **One:** it repeats itself. **Two:** it took minutes, when training ran at 200,000 tokens per second on the same hardware.

The second is a compilation problem. `fit()` and `predict()` compile the per-batch computation — `keras.ops` calls are lifted out of Python and optimised by the backend. Calling the model **directly** runs the forward pass live and **unoptimized at every step**.

Pad the input so the shape never changes

listing 16.10 PYTHON

```python
def compiled_generate(prompt, max_length=64):
    tokens = list(ops.convert_to_numpy(tokenizer(prompt)))
    prompt_length = len(tokens)
    tokens = tokens + [0] * (max_length - prompt_length)
    for i in range(prompt_length, max_length):
        prediction = mini_gpt.predict(np.array([tokens]), verbose=0)
        prediction = prediction[0, i - 1]
        tokens[i] = np.argmax(prediction).item()
    return tokenizer.detokenize(tokens)
```

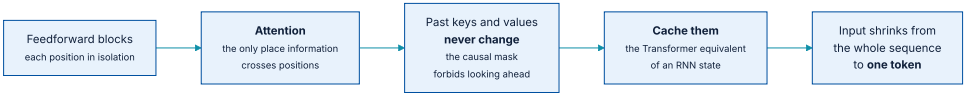
OUTPUT

```
>>> import timeit
>>> tries = 10
>>> timeit.timeit(lambda: compiled_generate(prompt), number=tries) / tries
0.48664706489998 93
```

From minutes to **under half a second**. Same maths, same weights, same output — a pure engineering win.

One inefficiency remains. Can you spot it?

With the RNN in chapter 15 we kept the state and computed one token at a time. A causal Transformer has an equivalent notion of state — you just have to look for it.



Attention is the only place information crosses positions, and past keys and values never change.

Key-value caching, and why every real system has it

Cache every key and value at every layer, and you have the Transformer equivalent of an RNN's state.

> Model input goes from *as long as the output* to **one token**. On a sequence thousands of tokens long this is a **thousandfold speed-up**. Any efficient implementation of generative sampling includes it.

Implementing it is clunky — saving and reusing intermediate arrays from every attention layer — which is exactly why you should use a library that has already done it.

SECTION 16.2.4

The repetition is not a bug

The model predicts the most likely next token across a billion words on many topics. Where there is no obvious continuation, an effective strategy is to guess **common words or repeated patterns** — and the model learns this almost immediately.

> Stop training very early and the model would generate the word `"the"` incessantly, because *the* is the most common word in English. Repetition is the objective **working correctly**, being asked the wrong question.

The output is a distribution, not a token

Taking the most likely output at each step is called **greedy search**. It is the most straightforward use of the predictions, and hardly the only one. Adding randomness lets us explore the learned distribution more broadly, and keeps us out of high-probability loops.

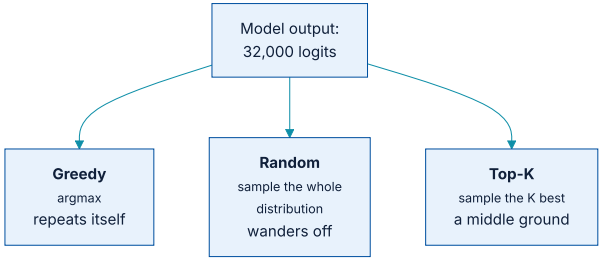


Figure 16.3 — three strategies over the same distribution.

Refactor: make the strategy a parameter

PYTHON

```python
def compiled_generate(prompt, sample_fn, max_length=64):
    tokens = list(ops.convert_to_numpy(tokenizer(prompt)))
    prompt_length = len(tokens)
    tokens = tokens + [0] * (max_length - prompt_length)
    for i in range(prompt_length, max_length):
        prediction = mini_gpt.predict(np.array([tokens]), verbose=0)
        prediction = prediction[0, i - 1]
        next_token = ops.convert_to_numpy(sample_fn(prediction))
        tokens[i] = np.array(next_token).item()
    return tokenizer.detokenize(tokens)

def greedy_search(preds):
    return ops.argmax(preds)
```

The sampling strategy is now a **first-class control** rather than a hardcoded `argmax` buried in a loop — which is how every serving framework exposes it too.

Sample the distribution directly

PYTHON

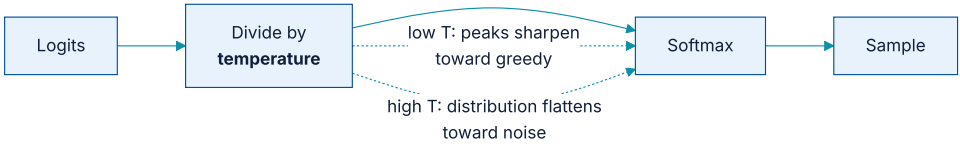
```python
def random_sample(preds, temperature=1.0):
    preds = preds / temperature
    return keras.random.categorical(preds[None, :], num_samples=1)[0]
```

OUTPUT

```
>>> compiled_generate(prompt, random_sample)
A piece of advice, just read my knees and stick with getables and a hello to me.
However, the bar napkin doesn't last as long. I happen to be waking up close and
pull it up as I wanted too and I still get it, really, shouldn't be a reaction
```

More diverse, and no longer stuck in loops — but now it **explores too much**. The output jumps around without continuity. We have traded one failure for its opposite.

One number that sharpens or flattens the distribution



Temperature acts on the logits, before the softmax — not on the probabilities after it.

- **Low temperature** makes all logits larger before the softmax, so the most likely output becomes even more likely.
- **High temperature** makes them smaller, so the distribution spreads out.

T = 2.0: not English any more

OUTPUT

```
>>> from functools import partial
>>> compiled_generate(prompt, partial(random_sample, temperature=2.0))
A piece of advice tran writes using ignore unnecessary pivot - come without
introdu accounts indicugel per divuren sendSolis silen om transparent
Gill Guide pover integer song arrays coding LIST**...Allow index criteria
Draw Reference Ex artifactincluding lib tak Br basunker increases entirelytembre
Any TextView cardinal spiritual heavenToen
```

Subword fragments, stray identifiers, tokens from other languages. At high temperature the distribution is flat enough that rare tokens win regularly, and **rare tokens in a 32,000 vocabulary are mostly debris**.

T = 0.8 and T = 0.2: fluency against repetition

OUTPUT

```
>>> compiled_generate(prompt, partial(random_sample, temperature=0.8))
A piece of advice I wrote about the same thing today. I have been a writer for
two years now. I am writing this blog and I just wrote about it. I am writing
this blog and it was really interesting.
```

OUTPUT

```
>>> compiled_generate(prompt, partial(random_sample, temperature=0.2))
A piece of advice, and a lot of people are saying that they have to be careful
about the way they think about it.
I think it's a good idea to have a good understanding of the way you think about
it.
I think it's a good idea to have a good understanding of the
```

At low temperature the behaviour converges on **greedy search**, repeating patterns. Temperature is a dial between two known failure modes, not a fix for either.

Restrict the choice to the K most likely tokens

PYTHON

```python
def top_k(preds, k=5, temperature=1.0):
    preds = preds / temperature
    top_preds, top_indices = ops.top_k(preds, k=k, sorted=False)
    choice = keras.random.categorical(top_preds[None, :], num_samples=1)[0]
    return ops.take_along_axis(top_indices, choice, axis=-1)
```

Temperature and top-K are not the same knob. A low temperature makes likely tokens more likely but rules **nothing** out. Top-K sets the probability of everything outside the K candidates to **zero**

K = 5, K = 20, and the two combined

OUTPUT

```
>>> compiled_generate(prompt, partial(top_k, k=5))
A piece of advice that I can't help it. I'm not going to be able to do anything
for a few months, but I'm trying to get a little better. It's a little too much.

>>> compiled_generate(prompt, partial(top_k, k=20))
A piece of advice and guidance from the Audi Bank in 2015. With all the above,
it's not just a bad idea, but it's very good to see that is going to be a great
year for you in 2017.
```

OUTPUT

```
>>> compiled_generate(prompt, partial(top_k, k=5, temperature=0.5))
A piece of advice that you can use to get rid of the problem.
The first thing you need to do is to get the job done. It is important that you
have a plan that will help you get rid of it.
```

The two compose: sampling the top five candidates at a temperature of 0.5 is a common production default.

Sampling is a control, and there are more of them

STRATEGY	WHAT IT DOES	FAILURE MODE
Greedy	Takes the argmax at every step	Repeats phrases indefinitely
Random	Samples the full categorical distribution	Wanders, no continuity
Temperature	Scales logits before the softmax	A dial between the two above
Top-K	Zeroes everything outside K candidates	K too small collapses to greedy
Beam search	Keeps several candidate chains alive at once	Expensive; can still be bland

With top-K our mini-GPT produces plausible English — with **little apparent utility**. That matches the GPT-1 result exactly: the generated output was a curiosity, and the state-of-the-art numbers came from fine-tuned classifiers.

What separates this from a modern LLM

100×
more parameters needed

1,000×
more training steps needed

And that is the entire gap. **The training recipe we just used is the exact blueprint everyone training LLMs uses today.**

The only missing pieces are a very large compute budget and some tricks for training across multiple machines — which chapter 18 covers. If we spent it, we would see the same leaps in quality OpenAI observed.

04

Using a pretrained LLM

A billion parameters, an instruction dataset, and a memory problem.

SECTION 16.3

Why almost nobody pretrains

1.3 million kWh
estimated to train Llama 2

45,000
American households' daily power

smaller than GPT-3
and this is the cheap case

Generative model training is now a **large percentage of total data-centre power consumption** at big technology companies. If every organisation using an LLM pretrained its own, the energy use would be a noticeable percentage of **global consumption**.

Loading Gemma 3, one billion parameters

listing 16.11 PYTHON

```python
gemma_lm = keras_hub.models.CausalLM.from_preset(
    "gemma3_1b",
    dtype="float32",
)
```

`CausalLM` is a high-level task API, like the `ImageClassifier` and `ImageSegmenter` of earlier chapters. It combines a tokenizer and a correctly initialised architecture into one Keras model, and loads matching weights.

> You must accept the **Gemma Terms of Use** on Kaggle before the weights will download, and authenticate with `kagglehub.login()`. The terms prohibit uses such as generating spam or hate speech — licence terms like this are becoming standard.

SECTION 16.3.1

Where a billion parameters actually sit

OUTPUT

```
>>> gemma_lm.summary()
Preprocessor: "gemma3_causal_lm_preprocessor"
| gemma3_tokenizer (Gemma3Tokenizer)  |  Vocab size: 262,144 |

Model: "gemma3_causal_lm"
| padding_mask (InputLayer)  | (None, None)          |           0 |
| token_ids (InputLayer)     | (None, None)          |           0 |
| gemma3_backbone            | (None, None, 1152)    | 999,885,952 |
| token_embedding            | (None, None, 262144)  | 301,989,888 |
|  (ReversibleEmbedding)     |                       |             |
 Total params: 999,885,952 (3.72 GB)
```

Two things worth reading closely. The vocabulary is **262,144** terms, eight times ours. And `ReversibleEmbedding` is the **tied embedding** we built by hand — 302 million of the billion parameters, counted once because input and output share them.

SECTION 16.3.1

It is a fancy autocomplete for the internet

OUTPUT

```
>>> gemma_lm.compile(sampler="greedy")
>>> gemma_lm.generate("A piece of advice", max_length=40)
A piece of advice from a former student of mine:
<blockquote>"I'm not sure if you've heard of it, but I've been told that the
best way to learn

>>> gemma_lm.generate("How can I make brownies?", max_length=40)
How can I make brownies?
[User 0001]
I'm trying to make brownies for my son's birthday party. I've never made
brownies before.
```

Far more coherent than mini-GPT — and **still not useful**. Asked how to make brownies, it wrote the *forum post asking the question*, because that is a likely continuation in crawled web text.

Change the prompt, change which part of the distribution you visit

OUTPUT

```
>>> gemma_lm.generate(
>>>     "The following brownie recipe is easy to make in just a few "
>>>     "steps.\n\nYou can start by",
>>>     max_length=40,
>>> )
The following brownie recipe is easy to make in just a few steps.

You can start by melting the butter and sugar in a saucepan over medium heat.
Then add the eggs and vanilla extract
```

It is tempting to imagine the model *interpreting* the prompt conversationally. **Nothing of the sort is going on.** We constructed a prompt for which an actual recipe is a more likely continuation than someone asking for help.

Prompt engineering is useful and hard to control

> *If this all sounds a bit hand-wavy and hard to control,* ***that's a good assessment.*** *Attempting to visit different parts of a model's distribution through prompting is often useful, but predicting how a model will respond to a given prompt is very difficult.*
>
> Section 16.3.1

Treat prompts as **empirical artefacts**: version them, test them against a fixed evaluation set, and expect them to break when the model behind them changes.

There is always a most-likely next token

OUTPUT

```
>>> gemma_lm.generate(
>>>     "Tell me about the 542nd president of the United States.",
>>>     max_length=40,
>>> )
Tell me about the 542nd president of the United States.
The 542nd president of the United States was James A. Garfield.
```

Utter nonsense — and the model could not find a **more likely** way to complete the prompt. There is no mechanism by which it could decline; declining is itself just another continuation, and one this model was never trained to prefer.

> Hallucination and uncontrollable output are **fundamental** problems with language models. If there is a silver bullet, **we have yet to find it**.

SECTION 16.3.2

Instruction fine-tuning: bend the output, do not relearn the language

Then train with the **same next-token loss** used for pretraining. The precise markup does not matter, as long as it is consistent.

> We are not learning a latent space for language here — that was done over trillions of tokens. We are **nudging the learned representation** to control the tone and content of the output.

Dolly: 15,000 handwritten instruction-response pairs

listing 16.12 PYTHON

```python
import json

PROMPT_TEMPLATE = "[instruction]\n{}[end]\n[response]\n"
RESPONSE_TEMPLATE = "{}[end]"

dataset_path = keras.utils.get_file(
    origin=(
        "https://hf.co/datasets/databricks/databricks-dolly-15k/"
        "resolve/main/databricks-dolly-15k.jsonl"
    ),
)

data = {"prompts": [], "responses": []}
with open(dataset_path) as file:
    for line in file:
        features = json.loads(line)
        if features["context"]:
            continue
        data["prompts"].append(PROMPT_TEMPLATE.format(features["instruction"]))
        data["responses"].append(RESPONSE_TEMPLATE.format(features["response"]))
```

Examples carrying extra **context** are discarded here — those are the RAG-shaped ones, which return later.

The template gives the examples a predictable shape

OUTPUT

```
>>> data["prompts"][0]
[instruction]
Which is a species of fish? Tope or Rope[end]
[response]

>>> data["responses"][0]
Tope[end]
```

Gemma is not a sequence-to-sequence model like our translator. But by training on prompts with this structure and generating only what follows the `[response]` marker, we can **use it in a sequence-to-sequence setting**.

PYTHON

```
ds = tf.data.Dataset.from_tensor_slices(data).shuffle(2000).batch(2)
val_ds = ds.take(100)
train_ds = ds.skip(100)
```

Batch size **2**. That number is about to become important.

SECTION 16.3.2

Inside the preprocessor

OUTPUT

```
>>> preprocessor = gemma_lm.preprocessor
>>> preprocessor.sequence_length = 512
>>> batch = next(iter(train_ds))
>>> x, y, sample_weight = preprocessor(batch)
>>> x["token_ids"].shape
(2, 512)
>>> x["padding_mask"].shape
(2, 512)
>>> y.shape
(2, 512)
>>> sample_weight.shape
(2, 512)
```

The padding mask tracks which inputs are padded zeros. The `sample_weight` tensor restricts the loss to **response tokens** — we do not care about loss on a fixed user prompt, and certainly not on padding.

SECTION 16.3.2

It is the same language-model setup underneath

OUTPUT

```
>>> x["token_ids"][0, :5], y[0, :5]
(Array([     2, 77074, 22768, 236842,    107], dtype=int32),
 Array([ 77074, 22768, 236842,    107,  24249], dtype=int32))
```

Each label is the next token value. **Instruction tuning is not a new objective** — it is the pretraining objective, run on carefully chosen data with the loss masked to the parts we care about.

SECTION 16.3.3

Why fit() would run out of memory right now

WHAT HAS TO BE IN MEMORY	SIZE
Model weights	**3.7 GB**
Adam: gradients, velocity, momentum — 3 floats per parameter	**~11 GB**
Intermediate values in the forward pass	a few GB
Total	**over 16 GB**

This is the common shape of LLM training: because parameter counts are so large, **GPU throughput is a secondary concern to fitting the model in accelerator memory at all**.

SECTION 16.3.3

Freezing saves optimizer state, not just gradients

Researchers have experimented extensively with which parameters to leave unfrozen in a Transformer. The answer, perhaps intuitively, is the ones in the **attention mechanism**.

But the attention layers still hold hundreds of millions of parameters. **Can we do better?**

Freeze the kernel, learn a low-rank correction

PYTHON

```python
class Linear(keras.Layer):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.kernel = self.add_weight(shape=(input_dim, output_dim))

    def call(self, inputs):
        return ops.matmul(inputs, self.kernel)
```

LoRA freezes `kernel` and adds a **low-rank decomposition** of the update alongside it: two matrices projecting down to an inner rank and back out.

Alpha and beta, and a rank in between

PYTHON

```python
class LoraLinear(keras.Layer):
    def __init__(self, input_dim, output_dim, rank):
        super().__init__()
        self.kernel = self.add_weight(
            shape=(input_dim, output_dim), trainable=False
        )
        self.alpha = self.add_weight(shape=(input_dim, rank))
        self.beta = self.add_weight(shape=(rank, output_dim))

    def call(self, inputs):
        frozen = ops.matmul(inputs, self.kernel)
        update = ops.matmul(ops.matmul(inputs, self.alpha), self.beta)
        return frozen + update
```

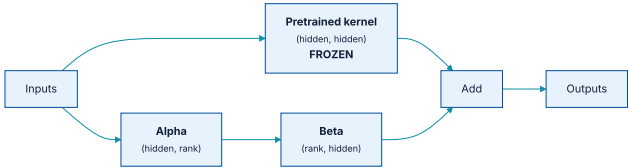


Figure 16.4 — the low-rank decomposition holds far fewer parameters than the kernel it corrects.

Four million parameters become thirty-two thousand

2048 × 2048
kernel shape

4,194,304
frozen parameters

32,768
trainable, at rank 8

The update does **not** have the expressive power of the original kernel — at the narrow middle, the entire update must pass through eight floats.

> That is the bet, and it holds: **during fine-tuning you no longer need the expressive power you needed during pre-training.** The representation is already built; you are only steering it.

In KerasHub it is one line

listing 16.13 PYTHON

```python
gemma_lm.backbone.enable_lora(rank=8)
```

The same thing written out layer by layer, which is what you would edit to add trainable parameters elsewhere:

the verbose equivalent PYTHON

```python
gemma_lm.backbone.trainable = False
for i in range(gemma_lm.backbone.num_layers):
    layer = gemma_lm.backbone.get_layer(f"decoder_block_{i}")
    layer.attention.key_dense.trainable = True
    layer.attention.key_dense.enable_lora(rank=8)
    layer.attention.query_dense.trainable = True
    layer.attention.query_dense.enable_lora(rank=8)
```

Written out this way you can see the two decisions the one-liner makes for you: **which layers** get LoRA, and at **what rank**. Adding it to the value projection, or to only the later blocks, is a one-line change from here.

A thousandfold cut in trainable parameters

OUTPUT

```
>>> gemma_lm.summary()
 Total params:         1,001,190,528 (3.73 GB)
 Trainable params:         1,304,576 (4.98 MB)
 Non-trainable params:   999,885,952 (3.72 GB)
```

The weights still occupy 3.7 GB. But the **trainable** parameters are now 5 MB — which takes the optimizer state from many gigabytes down to megabytes.

Note that total parameters went *up* slightly, from 999.9 M to 1,001.2 M: the alpha and beta matrices are new weights. **LoRA adds parameters to the model and removes them from the optimizer** — and the optimizer was the problem.

SECTION 16.3.3 · LISTING 16.14

Now the fine-tuning fits

listing 16.14 PYTHON

```python
gemma_lm.compile(
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=keras.optimizers.Adam(5e-5),
    weighted_metrics=[keras.metrics.SparseCategoricalAccuracy()],
)
gemma_lm.fit(train_ds, validation_data=val_ds, epochs=1)
```

We reach **55%** accuracy at guessing the next word of the response, against 36% for mini-GPT. `weighted_metrics` is what makes the `sample_weight` mask reach the metric, so the score is over response tokens only.

It answers questions now

OUTPUT

```
>>> gemma_lm.generate(
...     "[instruction]\nWhat is a proper noun?[end]\n[response]\n",
...     max_length=512,
... )
[instruction]
What is a proper noun?[end]
[response]
A proper noun is a word that refers to a specific person, place, or thing.
Proper nouns are usually capitalized and are used to identify specific
individuals, places, or things. Proper nouns are often used in formal writing
and are often used in titles, such as "The White House" or "The Eiffel
Tower."[end]
```

Much better. The model now **responds** to a question rather than carrying on the thought of the prompt text — and it emits `[end]` when it is finished, because the training data taught it where responses stop.

Have we solved hallucination? Not at all.

OUTPUT

```
>>> gemma_lm.generate(
...     "[instruction]\nWho is the 542nd president of the United States?[end]\n"
...     "[response]\n",
...     max_length=512,
... )
[instruction]
Who is the 542nd president of the United States?[end]
[response]
The 542nd president of the United States was James A. Garfield.[end]
```

Identical nonsense, now delivered in a helpful tone. **Instruction tuning changed the format of the answer, not its relationship to truth.**

> One thing that helps: train on many instruction/response pairs where the desired response is *"I don't know"* or *"As a language model, I cannot help you with that"*. This teaches the model to avoid whole topics where it would answer badly — **a behaviour, not an understanding**.

05

Going further with LLMs

RLHF, multimodality, retrieval, and models that show their working.

SECTION 16.4.1

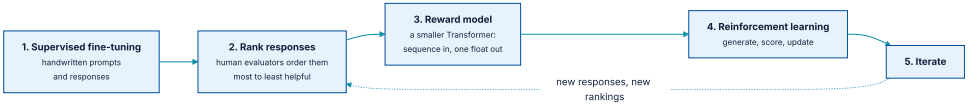
Handwritten data has a ceiling, and it is human

- Manually written examples are **slow and expensive** to come by, and will almost always become the bottleneck.
- The approach is limited by the **human performance ceiling** on the task. To beat human performance, we cannot supervise with human-written output.

The real thing to optimise is our **preference** for some responses over others. With a large enough sample of people this is perfectly well defined — the difficulty is translating *our preferences* into a loss function. That is what RLHF attempts.

SECTION 16.4.1

RLHF in five steps



The reward model is a proxy for human preference — a smaller Transformer that reads a sequence and outputs one float.

Responses to be ranked may be handwritten, generated by the model, or even written by other chatbots. Collecting rankings is expensive and slow — but **still faster than writing every desired response by hand**.

The simplest possible RLHF algorithm

listing 16.15 PYTHON

```python
for prompts in dataset:
    responses = model.generate(prompts)
    rewards = reward_model.predict(responses)
    good_responses = []
    for response, score in zip(responses, rewards):
        if score > cutoff:
            good_responses.append(response)
    model.fit(good_responses)
```

In practice you would **not discard** the bad responses — a bad response is a good signal about what not to do — and you would use specialised gradient updates over all responses. OpenAI used this setup to go from GPT-3's pretrained parameters to the first ChatGPT.

An instruction-tuned checkpoint, four times larger

listing 16.16 PYTHON

```python
gemma_lm = keras_hub.models.CausalLM.from_preset(
    "gemma3_instruct_4b",
    dtype="bfloat16",
)
```

float32 for training

4 bytes per parameter. 32-bit floats represent very small values, which helps keep training **gradients stable**.

bfloat16 for inference

2 bytes per parameter. No training here, so no gradient-stability concern — and **many gigabytes saved**.

SECTION 16.4.1

The template must match what the tuning used

PYTHON

```
PROMPT_TEMPLATE = (
    "<start_of_turn>user\n"
    "{}<end_of_turn>\n"
    "<start_of_turn>model\n"
)
```

OUTPUT

```
>>> prompt = "Why can't you assign values in Jax tensors? Be brief!"
>>> gemma_lm.generate(PROMPT_TEMPLATE.format(prompt), max_length=512)
<start_of_turn>model
Jax tensors are designed for efficient automatic differentiation. Directly
assigning values disrupts this process, making it difficult to track gradients
correctly. Instead, Jax uses operations to modify tensor values, preserving the
differentiation pipeline.<end_of_turn>
```

Pretrained on **14 trillion tokens**, then extensively tuned — supervised fine-tuning, RLHF, and distillation from a larger *teacher* model.

It refuses the bait — and then invents the date

OUTPUT

```
>>> prompt = "Who is the 542nd president of the United States?"
>>> gemma_lm.generate(PROMPT_TEMPLATE.format(prompt), max_length=512)
<start_of_turn>model
This is a trick question! As of today, November 2, 2023, the United States has
only had 46 presidents. There hasn't been a 542nd president yet.
You're playing with a very large number!<end_of_turn>
```

This is **not a new modelling technique**. It is the result of extensive training on trick questions like this one, with responses like this one.

> And look closely: having refused to hallucinate a president, the model **made up today's date**. Removing hallucinations is **whack-a-mole**, and this example is the demonstration.

SECTION 16.4.2

Multimodality is a sequence problem

The Transformer is not a text-specific model. It is a highly effective model for learning patterns in **sequence data**.

So if we can coerce another data type into a sequence representation, we can feed it to a Transformer and train on it. That is the whole idea.

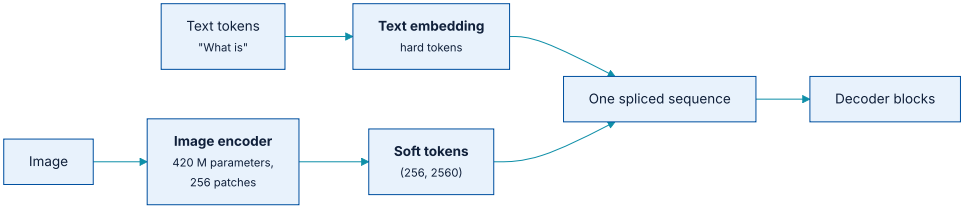


Figure 16.6 — hard text tokens and soft image tokens spliced into one sequence.

An image becomes 256 tokens that were never in a vocabulary

	HARD TOKENS	SOFT TOKENS
Source	A token ID	The vision encoder's output
Possible values	One row of the embedding matrix	**Any vector at all**
Count here	As many as the text has	256 per image
Loss	Trained to predict them	Loss is **zeroed** at these positions

Each image embeds as a **(256, 2560)** sequence, spliced into the text sequence after the token embedding layer.

SECTION 16.4.2

Asking questions about a picture

PYTHON

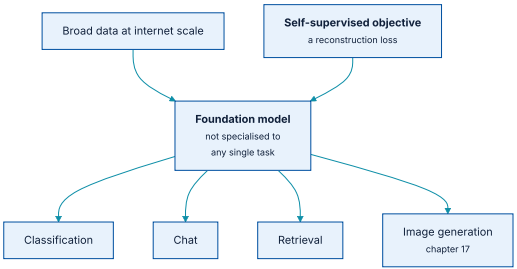
```
gemma_lm.preprocessor.max_images_per_prompt = 1
gemma_lm.preprocessor.sequence_length = 512

prompt = "What is going on in this image? Be concise!<start_of_image>"
gemma_lm.generate({
    "prompts": PROMPT_TEMPLATE.format(prompt),
    "images": [image],
})
```

OUTPUT

```
<start_of_turn>model
A snake wearing glasses is sitting in a leather armchair, surrounded by a large
bookshelf, and reading a book. It's a whimsical, slightly surreal image.
<end_of_turn>
```

The special token `<start_of_image>` is expanded into 256 placeholder positions, which are then **replaced** by the soft tokens from the vision encoder.

Foundation models



The hallmarks: a self-supervised reconstruction loss, and no specialisation to a single task.

This is a **striking and recent shift**. Rather than training from scratch on your own dataset, you are often better off getting a rich representation from a foundation model and specialising it — with the downside of running billions of parameters, which is **hardly a fit for every application**.

Two reasons an LLM is not a search engine

It makes things up

False *facts* that were not in the training data but could be interpolated from it. This ranges from **misleading to dangerous**.

Its knowledge has a cutoff

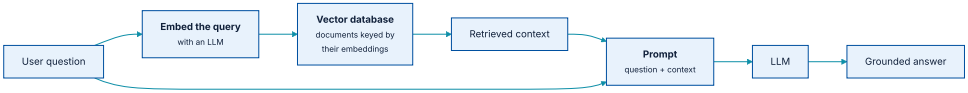
At best, the date it was pretrained. Training is expensive and cannot run continuously, so knowledge of the world just **stops**.

Nobody wants a search engine that can only tell you about things that happened six months ago. But if we treat the LLM as **conversational software** that can handle any sequence data in a prompt, we can use it as the **interface** to information retrieved by more traditional search.

Retrieve first, then generate

OUTPUT

```
Use the following pieces of context to answer the question.

Question: What are some good ways to improve sleep?

Context: {text from a medical journal on improving sleep}

Answer:
```



A vector database keys documents by their embeddings and returns the nearest ones to the query embedding.

Three problems it addresses, and one it does not

1. It gives an obvious way to **work around the cutoff date** of the model.
2. It lets the model access **private data** — a company can use a publicly-trained LLM as an interface to information it holds internally.
3. It helps **factually ground** the model: an LLM is much less likely to invent facts on a topic when correct context sits in the prompt.

> There is **no silver bullet that stops hallucination entirely.** RAG makes it less likely on topics you retrieved context for. It does nothing for topics you did not.

One pleasing detail: the vector database's *query*, *key*, and *value* vocabulary is not an accident — attention borrowed those terms **from database systems** in the first place.

SECTION 16.4.4

Grade-school maths defied progress for years

For most NLP problems the recipe was easy: more data, better benchmark score. Grade-school word problems did not respond to it.

> *In 2023 researchers at Google noticed that prompting the model with a few examples of showing your work made the model do the same — and that by attending to its own written-out steps, it reached correct solutions far more often. They called it* ***chain-of-thought prompting****.*
>
> Section 16.4.4

Another group found the examples were not even necessary: prompting with *"Let's think step by step"* was enough.

Chain-of-thought fine-tuning, in five steps

1. Collect basic maths and reasoning problems with their desired answers.
2. Generate, **with randomness**, a number of responses to each.
3. Find responses with a correct answer by **string parsing** — prompt the model to mark its final answer with a specific token.
4. Run supervised fine-tuning on the correct responses, **including all the intermediate output**.
5. Repeat.

The answer check acts as the **environment**; the generated outputs are the **actions**. In practice you would use a more complex gradient step that learns from incorrect responses too.

The same prompt, twice, with random sampling

OUTPUT

```
FIRST ATTEMPT
* Letters per week:  3 friends * 2 letters/week = 6 letters/week
* Letters per month: 6 letters/week * 4 weeks/month = 24 letters/month
* Letters in 3 months: 24 letters/month * 3 months = 72 letters
* Total letters: 72 letters * 2 = 144 letters
ANSWER: 144
```

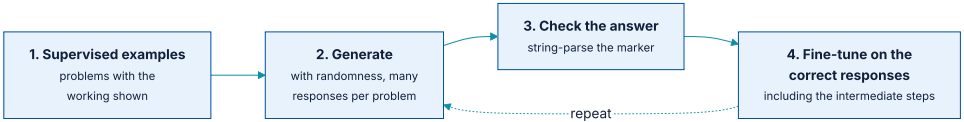
OUTPUT

```
SECOND ATTEMPT
* Letters per week:  3 friends * 2 letters/week = 6 letters/week
* Letters per month: 6 letters/week * 4 weeks/month = 24 letters/month
* Total letters: 24 letters/month * 3 months = 72 letters
ANSWER: 72
```

The first attempt got hung up on the **superfluous detail** that each letter has two pages. The second is right. Same model, same prompt, different sample — which is exactly why step 3 of the recipe is *check the answer*.

SECTION 16.4.4

Where else verifiable answers exist



The loop only closes where correctness can be checked automatically.

The same idea applies wherever a text prompt has an obvious, **verifiable** answer. **Coding** is the important one: prompt the model for code, then actually run it to test the quality of the response.

One trend is clear across all these domains: as a model learns harder questions, it spends **more and more time showing its work** before answering. Think of it as learning to search over its own potential solutions.

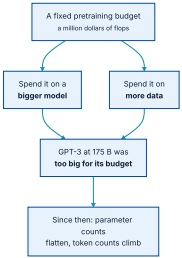
06

Where are LLMs heading next?

More parameters is the obvious answer. It is not quite the right one.

SECTION 16.5

A fixed budget buys flops, and you must choose how to spend them



Recent research found GPT-3 was far too big for the compute it was given.

Training a **smaller model on more data** would have produced better performance. So model sizes have trended flatter while data sizes have trended up.

SECTION 16.5

Two reasons parameter counts stopped climbing

GPT-3 was undertrained

175 billion parameters against roughly half a trillion tokens. We now know the balance was wrong.

Deployment cost

It is often worth sacrificing performance for a smaller model that fits cheaper hardware. **A really good model does not help if it is prohibitively expensive to run.**

This does **not** mean scaling has stopped. More compute does generally give better performance, and we have yet to see any sign of an asymptote where next-token prediction levels off. Billions of dollars continue to go into finding out what emerges.

We are starting to run out of pretraining data

> Models are starting to *eat their own tail* — training on a significant portion of content created by other LLMs, which brings a whole other set of concerns.

This is one reason reinforcement learning is drawing so much attention. **If you can build a difficult, self-contained environment that generates new problems, you can keep training on the model's own output** — with no need to scrounge the web for more quality text.

None of this is a silver bullet

> *At the end of the day, the fundamental problem remains that LLMs are **wildly inefficient at learning compared to humans**. Model capabilities only come from training on many orders of magnitude more text than people will read in their lifetimes.*
>
> Section 16.5

As scaling continues, so will more fundamental research into models that learn quickly from limited data.

Still — LLMs represent the ability to build **fluent natural language interfaces**, and that alone brings a massive shift in what can be accomplished with computing devices. This chapter laid out the basic recipe by which they get there.

COMMON FAILURE MODES

Four ways LLM work goes wrong in practice

Uncompiled or reshaping generation

Calling the model directly, or letting the input shape change each step, costs **two to three orders of magnitude** in speed. No error is raised — only a bill.

Prompt template mismatch

An instruction-tuned checkpoint expects the exact template it was tuned with. Use a different one and quality drops sharply, silently.

Full fine-tuning on a small GPU

Adam needs three extra floats per parameter. The model loads and generates fine, then `fit()` dies on the first step. Use LoRA.

Mistaking fluency for correctness

Instruction tuning and RLHF change the **form** of an answer. Neither makes it true. Ground with retrieval, and verify where you can.

SUMMARY

What has to survive this chapter (1 of 2)

1. **An LLM is three ingredients**: the Transformer architecture, a language modelling task, and a large amount of unlabelled text.
2. **GPT is decoder-only** — expressivity traded for an unbounded supply of training data, and the data won.
3. **Tied embeddings** use one matrix forward at the input and transposed at the output, saving a large share of the parameters.
4. **Warmup** matters: large Transformers are sensitive to early updates, and exploding gradients are the common failure.
5. **Sampling is a separate design** from the model. Greedy repeats, random wanders, temperature dials between them, top-K rules tokens out.
6. **Key-value caching** turns generation from quadratic re-computation into one token per step.

SUMMARY

What has to survive this chapter (2 of 2)

1. **Instruction fine-tuning** is the pretraining objective on curated data with the loss masked to responses — it bends output, it does not relearn language.
2. **LoRA** freezes the kernel and learns a low-rank correction, cutting trainable parameters a thousandfold and optimizer memory with them.
3. **RLHF** replaces handwritten answers with ranked preferences and a reward model, which lifts the human performance ceiling.
4. **Multimodality is a sequence problem** — encode images as soft tokens and splice them into the text sequence.
5. **RAG** puts retrieved context in the prompt, addressing the knowledge cutoff and private data, and reducing but not removing hallucination.
6. **All LLMs hallucinate.** Fluency, helpfulness, and refusal are trainable behaviours; truth is not one of them.

NOTEBOOK
05_instruction_tuning_with_lora.ipynb

PAPER
Hu et al., LoRA (2021)

NEXT
Chapter 17 — Image generation